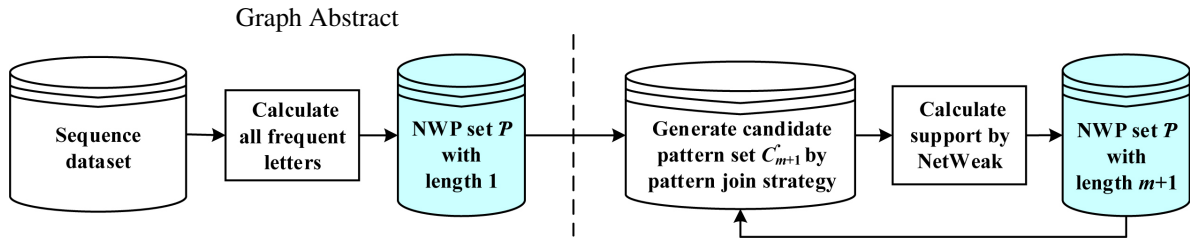


List of Figures

1	Comparison of matching results for time series data. The figures show that the time series in (a) is pattern “AfO” and those in (b)–(c) can be symbolized as “AOafAO” and “AGfcAEO”, respectively. The green and red symbols represent weak and strong fluctuations in the gaps, respectively. The trend of the time series in (a) is similar to that in (b), while it is significantly different from that in (c), since some strong fluctuations cannot be neglected. The reason is that there are strong fluctuations in gap constraints. For example, character ‘G’ as a strong character is matched by gap A[0,3]f, which makes it significantly different from its pattern. Therefore, strong characters cannot be matched by gap constraints.	5
2	Nettree for pattern p in sequence s created by NETGAP. The Nettree has four levels, since the pattern length is four. It also has four roots, n_1^1 , n_1^7 , n_1^8 and n_1^{13} , and four leaves, n_4^7 , n_4^8 , n_4^{13} and n_4^{19} . Some nodes have the same node ID. For example, both n_1^7 and n_4^7 have the same ID of 7. Nodes may have more than one parent. For example, node n_2^9 has two parents, n_1^7 and n_1^8	8
3	Nettree for pattern p with weak-gap in sequence s created by NETGAP. Although $s_9 = p_2 = \text{“a”}$ and $s_{11} = p_3 = \text{“e”}$, node n_3^{11} cannot be created, since s_{10} is “e”, a strong character that cannot be matched by the weak-gap a(0,2)e. Similarly, node n_4^{13} cannot be created since s_{11} is “e”.	8
4	Nettree of pattern p in sequence s created by NetWeak. The Nettree is simplified to a forest, since each node has at most one parent.	9
5	Framework of NWP-Miner	11
6	Comparison of running time with competitive algorithms	13
7	Comparison of number of candidate patterns with competitive algorithms	13
8	Comparison of number of nodes with competitive algorithms	14
9	Comparison of memory usage with competitive algorithms	14
10	Comparison of number of patterns	15
11	Comparison of running time	15
12	Comparison of number of candidate patterns	15
13	Comparison of memory usage	15
14	Comparison of mined NFPs and NWP	16
15	Comparison of matching results of pattern “AeO” on S9	16
16	Comparison of matching results of pattern “aBf” on S10	16



Highlights

- We limit the gap constraints to make it match weak characters to avoid noise patterns.
- We propose a complete and efficient algorithm NWP-Miner.
- NWP-Miner employs depth-first search and back-tracking strategies to calculate support.
- NWP-Miner has better performance than other competitive algorithms.
- **NWP-Miner discovers more meaningful patterns and filters out noise patterns.**

NWP-Miner: Nonoverlapping Weak-gap Sequential Pattern Mining

Youxi Wu^{a,c,d}, Zhu Yuan^a, Yan Li^{b,*}, Lei Guo^c, Philippe Fournier-Viger^e, Xindong Wu^{f,g}

^aSchool of Artificial Intelligence, Hebei University of Technology, Tianjin 300401, China

^bSchool of Economics and Management, Hebei University of Technology, Tianjin 300401, China

^cState Key Laboratory of Reliability and Intelligence of Electrical Equipment, Hebei University of Technology, Tianjin 300401, China

^dHebei Key Laboratory of Big Data Computing, Tianjin 300401, China

^eSchool of Humanities and Social Sciences, Harbin Institute of Technology (Shenzhen), Shenzhen, China

^fKey Laboratory of Knowledge Engineering with Big Data (Hefei University of Technology), Ministry of Education, Hefei 230009, China

^gMininglamp Academy of Sciences, Mininglamp Technology, Beijing 100084, China

Abstract

Nonoverlapping sequential pattern mining (SPM) is a type of SPM with gap constraints that can mine valuable information in sequences. One of the disadvantages of nonoverlapping SPM is that any characters can match with gap constraints. Hence, there can be a significant difference between the trend of a pattern and those of its occurrences. To tackle this issue, we propose nonoverlapping weak-gap sequential pattern (NWP) mining, where characters are divided into two types: weak and strong. This allows discovering frequent patterns more accurately by limiting the gap constraints to match only weak characters. To discover NWPs, we propose NWP-Miner which involves two key steps: support calculation and candidate pattern generation. To efficiently calculate the support of candidate patterns, depth-first search and backtracking strategies based on a simplified Nettoree structure are adopted, which effectively reduce the time and space complexities of the algorithm. Moreover, a pattern join approach is applied to effectively reduce the number of candidate patterns. The experimental results show that NWP-Miner is more efficient than other competitive algorithms. More importantly, the case study of time series shows that NWP-Miner can effectively filter out noise patterns and discover more meaningful patterns. Algorithms and datasets can be downloaded from <https://github.com/wuc567/Pattern-Mining/tree/master/NWP-Miner>.

Keywords: sequential pattern mining, frequent pattern, time series, gap constraint, depth-first search, Nettoree structure

1. Introduction

Sequential pattern mining (SPM) can be used to discover valuable subsequences (called patterns) from a large amount of data [45], and has been widely used in many fields, such as outlying sequence data analysis [35], communication networking [2], big data intelligence [15, 37], co-location mining [34], and e-commerce shopping analysis [17]. Classical SPM methods take an ordered lists of sets of items (characters) as input [27]. For example, in a transaction sequence $S = \langle a(ac)d(acd) \rangle$, it means that a customer purchased item “a” in the first time, items “a” and “c” in the second time, item “d” in the third time, and “a”, “c”, and “d” in the last time. In this example, we know that

each item is composed of a set, such as “(acd)”. Moreover, classical SPM methods generally ignore the fact that a pattern such as “ac” may appear multiple times in the sequence. However, in many cases, such as DNA, protein, virus, and time series, each item is composed of only an element, which means that a sequence is a list of single items [20]. For example, “attaaaggttt” is a segment of SARS-CoV-2. More importantly, it is worthy to consider a pattern that may appear multiple times in the sequence. Nevertheless, if we only mine continuous subsequences, we will get very little information. If we mine discontinuous subsequences, we will get an excessive number of patterns. To solve this problem, gap constraint SPM [50] was proposed, whose goal is to identify all subsequences that appear frequently by only counting occurrences that satisfy some user-defined gap constraints. A pattern $\mathbf{p}$ with gap constraints can be represented as $\mathbf{p} = p_1[a, b]p_2 \cdots [a, b]p_j \cdots [a, b]p_m$, where

*Corresponding author

Email address: lywuc@163.com (Yan Li)

$0 \leq a \leq b$, and a and b are the minimum and maximum number of wildcards between characters p_{j-1} and p_j , respectively [29]. Gap constraints allow users to set a particular gap based on their specific needs, which allows for a more flexible and targeted search. Thus, gap constraint SPM has been used in many different fields, such as biological sequence analysis [38], text corpus mining [26], and feature extraction [47].

Nonoverlapping SPM is a type of SPM with gap constraints that can effectively mine valuable information in sequences of characters [47], and NOSEP was proposed to mine frequent patterns [45]. Gap constraint $[a, b]$ can match any at least a characters and at most b characters without any restrictions [23]. **If the events concerned by users are matched by the gap constraint, it may result in a significant difference between the trend of a pattern and those of its occurrences.** For clarification, we use time series data as an example to illustrate this issue. If we use NOSEP to mine frequent patterns with gap constraint $[0,3]$ on the closing index of West Texas Intermediate (WTI) from January 1, 2019 to October 2, 2019, we can get a frequent pattern “A[0,3]f[0,3]O”, whose corresponding trend is shown in Fig. 1(a). We also select two occurrences of the pattern shown in Figs. 1(b) and (c) which are two fragments of the time series from April 26, 2019 to May 6, 2019, and from August 9, 2019 to August 20, 2019, respectively. **To observe the trend of time series, they are symbolized according to their gradients.** The gradient can be calculated according to $f_i = (t_{i+1} - t_i)/t_i$. Therefore, f_i can be symbolized as characters according to certain rules. For example, in the range of $[1\%, 2\%)$, f_i is symbolized as character ‘A’. In the range of $[2\%, 4\%)$, it is symbolized as character ‘B’. In the range of $[32\%, \text{inf})$, it is symbolized as character ‘F’. The corresponding negative range is symbolized as a lowercase. Therefore, the fragments of time series are symbolized as “AOafAO” and “AGfcAEO”, respectively. According to Nonoverlapping SPM, the trend in Fig. 1(a) should be both similar to those in Figs. 1(b) and (c), since “AOafAO” and “AGfcAEO” are two occurrences of the pattern. However, the trend of the time series in (a) is similar to that in (b), while it is significantly different from that in (c). The reason is as follows. The fluctuation of the time series in (a) is a small rise, followed by a large drop, and followed by a small rise. The fluctuation of the time series in (b) is similar to that in (a). However, the fluctuation of the time series in (c) is significantly different from that in (a), since it is a small rise, followed by a large rise, followed by a large drop, followed by a small drop, followed by a small rise, followed by a large rise, and followed by a small rise. This example shows

that whether the curves have the same trend is determined by strong characters rather than weak characters, where strong character corresponds to a large rise/drop in time series, while weak character corresponds to a small rise/drop. Hence, this example illustrates that strong characters cannot be ignored in gap constraints.

To avoid the phenomenon that a pattern with gap constraints that is significantly different from its occurrences, this paper proposes a novel task of nonoverlapping weak-gap sequential pattern (NWP) mining, which requires that the gap constraints can only be matched by weak characters. Since our method can avoid the phenomenon that NWPs are significantly different from their occurrences, the NWPs mined by our method are more accurate than the patterns mined by existing methods [3, 45]. The main contributions of this paper are as follows.

- (1) **To avoid noise patterns, we address the NWP mining which limits the gap constraints to make it match only weak characters, and propose a complete and efficient algorithm called NWP-Miner, which involves two key steps: support calculation and candidate pattern generation.**
- (2) **To efficiently calculate the support of patterns, NWP-Miner performs the depth-first search and backtracking strategies based on a simplified Net-tree structure, which reduce the time and space complexities. To generate candidate patterns, a pattern join strategy is applied which effectively reduces the number of candidate patterns.**
- (3) Experimental results on real-life time series datasets demonstrate that NWP-Miner not only gives better performance than other competitive algorithms, but can also effectively filter out noise patterns and discover more meaningful patterns.

The remainder of the paper is organized as follows. Related work is summarized in Section 2, and problem definition is given in Section 3. Section 4 describes the proposed NWP-Miner scheme and illustrates the principle of the algorithm through some examples. Section 5 reports the performance of NWP-Miner, and Section 6 concludes the paper.

2. Related work

SPM has been widely applied in many fields [9]. A variety of mining methods have been proposed to meet a range of different needs, such as negative SPM [4], closed SPM [5, 48], contrast SPM [12], periodic pattern mining [8], and high utility SPM [28, 32]. For example,

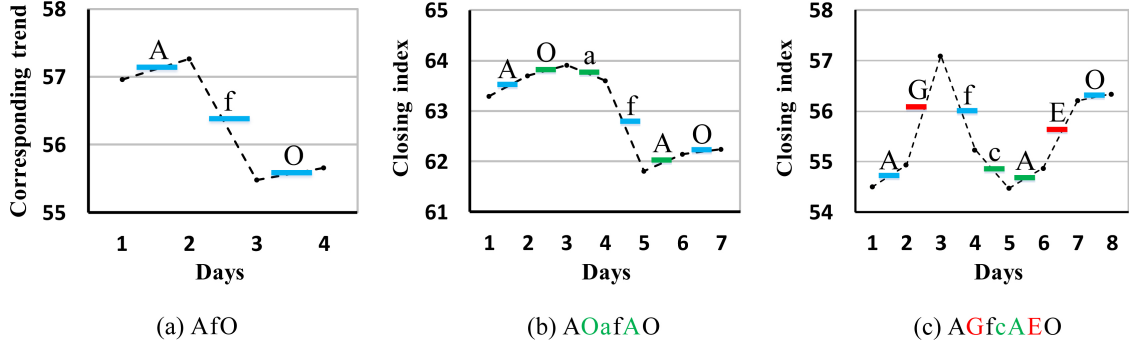


Fig. 1. Comparison of matching results for time series data. The figures show that the time series in (a) is pattern “AfO” and those in (b)–(c) can be symbolized as “AOafAO” and “AGfcAEO”, respectively. The green and red symbols represent weak and strong fluctuations in the gaps, respectively. The trend of the time series in (a) is similar to that in (b), while it is significantly different from that in (c), since some strong fluctuations cannot be neglected. The reason is that there are strong fluctuations in gap constraints. For example, character ‘G’ as a strong character is matched by gap A[0,3]f, which makes it significantly different from its pattern. Therefore, strong characters cannot be matched by gap constraints.

Qiu et al. [25] presented an efficient method for modeling nonoccurring behaviors by negative SPM. Bai et al. [1] explored an efficient incremental algorithm to discover historical moments in sequence data. Fournier-Viger et al. [6] proposed two efficient algorithms to mine significant trend sequence patterns from real social networks. Park et al. [24] presented an efficient algorithm called SAX-ARM that can find abnormal event patterns from multivariate time series. Gan et al. [10] explored an effective algorithm to discover high-utility patterns. Sumalatha et al. [30] proposed a distributed mining algorithm based on the MapReduce framework to effectively mine high-utility time interval sequence patterns from big data.

The classical SPM methods ignore the fact that some subsequences may occur many times in a sequence. To solve this problem, gap constraint SPM can mine the subsequences with gap constraint to meet the needs of users. Compared with classical SPM, gap constraint SPM is more challenging, since gap constraint SPM methods generally adopt pattern matching strategy to calculate the support [42, 43]. More importantly, when calculating the occurrence of a pattern, the constraints can be subdivided into three types of conditions: no condition [46, 50], the one-off condition [14, 38], and the nonoverlapping condition [33].

- (1) No condition means that any character in the sequence can be used many times, i.e. there are no constraints on the occurrences [46]. It is easy to calculate the support for no condition, but the support or support rate under no condition does not satisfy the Apriori property [50]. Thus, Apriori-like method is employed to mine all frequent patterns,

which enlarges the space of candidate patterns [18]. The advantage of Apriori-like method is that it is a complete method, but the drawback of this approach is that more candidate patterns are checked to mine all frequent patterns. To avoid expanding the scope of the candidate patterns, an approach is appending some unmatched characters at the end of the sequence [22]. Thus, the new method satisfies the Apriori property.

- (2) The one-off condition means that any character in the sequence can be used once at most [41]. The advantage of SPM under one-off condition is that it satisfies the Apriori property, since the support of subpattern is no less than that of its pattern [16]. Unfortunately, calculating the support under the one-off condition is an NP-complete problem, since it can be proved by the reduction of the iterative shuffle problem [36], which is NP-complete. Therefore, it is impossible to accurately calculate the support. Kinds of heuristic strategies are therefore employed to calculate the pattern support [11, 41].
- (3) The nonoverlapping condition means that any character in the sequence can be rematched by characters at different positions in the pattern [43, 47]. SPM under the nonoverlapping condition (or nonoverlapping SPM) is a complete mining method satisfying the Apriori property. Some nonoverlapping SPM methods have been proposed. For example, top-k self-adaptive contrast nonoverlapping SPM was explored in feature extraction of sequence classification to discover the patterns that are frequent in sequences with positive label but are rare in sequences with negative label. Li et al. [20] applied the maximal nonoverlapping SPM method to

analyze the pattern composition of the SARS-CoV-1, SARS-CoV-2, and MERS-CoV viruses in biological sequences. The results show that the three viruses are similar in the short patterns but different in the long patterns. **To mine the patterns with low frequency but high utility, high-average utility nonoverlapping SPM was proposed, which can be used in product recommendations [40].** Therefore, the method can help retailers achieve the maximum profits. However, one of the disadvantages is that a lot of frequent patterns can be mined, which contain many redundant short patterns. Thus, closed nonoverlapping SPM was explored to reduce the redundant short patterns [48].

Although nonoverlapping SPM can effectively extract interesting patterns, one of its disadvantages is that the mined patterns may contain noise, since the gap can match any character. For example, Fig. 1(c) which can be seen as an occurrence of Fig. 1(a) is significantly different from Fig. 1(a). **To solve this problem, we propose an NWP mining scheme that can mine weak-gap patterns more accurately, and eliminate such deviation.** Table 1 shows a comparison of related work in this area.

From Table 1, we know that references [21] and [49] are researches which do not consider the repetition of pattern in the sequences. Thus, gap constraints are not allowed in these researches. Therefore, these two studies are significantly different from this paper. References [18] and [46] are SPM under no condition, while this paper focuses on SPM under the nonoverlapping condition. Therefore, the support calculation methods are different. More importantly, the candidate pattern generation methods are different, since references [18] and [46] adopt Apriori-like property, while our paper employs Apriori property. Reference [16] is a SPM under the one-off condition. Thus, the support calculation method is different from ours. Since reference [16] cannot completely calculate the support, some feasible patterns may be lost. Therefore, it is an approximate mining method. However, our approach can completely mine all patterns. Reference [3] is also a nonoverlapping SPM, and adopts a heuristic strategy to calculate the support, resulting in the loss of some feasible patterns. Therefore, it is also an approximate mining method. References [31] and [45] represent the most relevant researches. The main difference between the scheme proposed here and the approach in [31] is that the former is designed for the case of nonoverlapping condition, while the latter is for no condition. Since the support or support rate under no condition does not satisfy the Apriori property, the scheme in [31] appends

some unmatched characters at the end of the sequence. In this way, the method can be made to satisfy the Apriori property, but it still remains an approximate mining method [31]. In contrast, our method employs the nonoverlapping condition, which is a complete mining method satisfying the Apriori property [45]. There are two main differences between this paper and reference [45]. Firstly, in [45], the gap can match any character, while in this paper, the gap can match only weak characters to eliminate deviations. Secondly, when calculating the support, the scheme in [45] creates a whole Ntree and deletes invalid nodes after obtaining an occurrence. **This paper presents a more effective method that adopts the depth-first search and backtracking strategies to find all occurrences, with no need to create the whole Ntree and delete invalid nodes.** Hence, the time complexity is improved.

In short, the current gap constraint SPM methods have the following disadvantage. Any characters can match with gap constraints, which will lead to a significant difference between the trend of the pattern and those of its occurrences. Therefore, many meaningless noise patterns are discovered. To solve this issue, this paper explores NWP mining and proposes the NWP-Miner algorithm to effectively discover more meaningful patterns.

3. Problem Definition

This section defines key concepts and formally introduces the problem of NWP mining.

Definition 1. (Sequence) A sequence with length n is described as $s = s_1 \cdots s_i \cdots s_n$, where $s_i (1 \leq i \leq n) \in \Sigma$, Σ represents the set of items, and the size of Σ is expressed as $|\Sigma|$.

It is worth noting that the definition of sequence in this paper is different from that in classical SPM. **In classical SPM, the sequence is defined as a series of sets of items.** However, the sequence in this paper is a set of items, and can be regarded as a special case of classical sequence, since each set has only an item. For example, $\langle a(ac)d(acd) \rangle$ is a classical sequence, since $\{ac\}$ and $\{acd\}$ are two sets of items. $\langle aacdacd \rangle$ is a sequence in this paper, and DNA, protein, virus, and time series are this type of sequence. We need to point out that NWP-Miner cannot mine NWPs in classical sequences. In the future, we will focus on mining NWPs in classical sequences.

Definition 2. (Sets of Weak and Strong Characters) For an application, if the user is not interested in character

Table 1. Related work

Research	Type of condition	Gap constraint	Mining type	Pattern type	Pruning strategy
Lin et al. [21]	Utility	None	Exact	Frequent	Apriori
Yun et al. [49]	Weighted	None	Exact	Maximal	Other
Li et al. [18]	No condition	Periodic gap	Exact	Long	Apriori-like
Wu et al. [46]	No condition	Periodic gap	Exact	Top- K	Apriori-like
Lam et al. [16]	One-off condition	Periodic gap	Approximate	Compressed	Apriori
Ding et al. [3]	Nonoverlapping condition	Periodic gap	Approximate	Closed	Apriori
Wu et al. [45]	Nonoverlapping condition	Periodic gap	Exact	Frequent	Apriori
Tan et al. [31]	No condition	Weak-gap	Approximate	Frequent	Apriori
This paper	Nonoverlapping condition	Weak-gap	Exact	Frequent	Apriori

t ($t \in \Sigma$), then t is a weak character and is stored in the weak character set Ω , otherwise it is a strong character and is stored in the strong character set Γ .

Definition 3. (Pattern with Gap and Pattern with Weak-Gap) A pattern $\mathbf{p}$ with gap is denoted as $\mathbf{p} = p_1[a, b]p_2 \cdots [a, b]p_j \cdots [a, b]p_m$ (or abbreviated as $\mathbf{p} = p_1p_2 \cdots p_m$ with $gap=[a, b]$), where $p_j \in \Sigma$ ($1 \leq j \leq m$), m is the pattern length, and a and b ($0 \leq a \leq b$) are integers indicating the minimum and maximum gaps between p_{j-1} and p_j , respectively. Accordingly, a pattern $\mathbf{p}$ with weak-gap is denoted by $\mathbf{p} = p_1(a, b)p_2 \cdots (a, b)p_j \cdots (a, b)p_m$ (or abbreviated as $\mathbf{p} = p_1p_2 \cdots p_m$ with $wgap = (a, b)$), where weak-gap (a, b) can match minimum a and maximum b weak characters in Ω between characters p_{j-1} and p_j .

Definition 4. (Occurrence and Nonoverlapping Occurrence) Suppose we have a sequence $\mathbf{s}$, a pattern $\mathbf{p}$ with weak-gap (a, b) and a character set Ω . $L = \langle l_1, l_2, \dots, l_m \rangle$ ($1 \leq l_1 < l_2 < \dots < l_m \leq n$) is an occurrence with weak-gap (hereafter, an ‘occurrence’) of pattern $\mathbf{p}$ in sequence $\mathbf{s}$, if and only if $p_1 = s_{l_1}$, $p_2 = s_{l_2}, \dots, p_m = s_{l_m}$, $a \leq l_j - l_{j-1} - 1 \leq b$ and $s_t \in \Omega$ ($l_{j-1} < t < l_j$). Suppose we have two occurrences $L = \langle l_1, l_2, \dots, l_m \rangle$ and $L' = \langle l'_1, l'_2, \dots, l'_m \rangle$ in the same sequence. L and L' are two nonoverlapping occurrences, if and only if for all j , $l_j \neq l'_j$ ($1 \leq j \leq m$). For a sequence database, if L and L' are two occurrences in different sequences, then the two occurrences are nonoverlapping.

Definition 5. (Support in a Sequence and Sequence Database) The number of nonoverlapping occurrences of a pattern $\mathbf{p}$ in a sequence $\mathbf{s}$ is called its support, and is denoted by $sup(\mathbf{p}, \mathbf{s})$. A sequence database with N sequences is denoted by $SDB = s_1, s_2, \dots, s_N$. The support of a pattern $\mathbf{p}$ in SDB is the sum of its support in each sequence, i.e. $sup(\mathbf{p}, SDB) = \sum_{k=1}^N sup(\mathbf{p}, \mathbf{s}_k)$.

Definition 6. (NWP) If $sup(\mathbf{p}, SDB)$ is no less than a user-defined minimum support threshold $minsup$, then pattern $\mathbf{p}$ is an NWP.

Example 1. Suppose we have $SDB = \{s_1 = s_1s_2s_3s_4s_5 = BCBCC, s_2 = s_1s_2s_3s_4s_5 = BDCBC, s_3 = s_1s_2s_3s_4s_5 = BBCAC\}$, $\Omega = \{A, B\}$ and $minsup = 2$. There are two nonoverlapping occurrences of pattern $\mathbf{p} = B(0,1)C(0,1)C$ in $s_1: \langle 1, 2, 4 \rangle$ and $\langle 3, 4, 5 \rangle$. There are no nonoverlapping occurrences of $\mathbf{p}$ in s_2 since the character ‘D’ is a strong character. There is only one occurrence of $\mathbf{p}$ in $s_3: \langle 1, 3, 5 \rangle$. Thus, $sup(\mathbf{p}, SDB) = 2 + 0 + 1 = 3 \geq minsup$. Hence, pattern $\mathbf{p}$ is an NWP.

Definition 7. (NWP Mining) Given an SDB , the set of weak characters Ω and a minimum support threshold $minsup$, the problem of NWP mining is to discover all NWPs.

The symbols used in this paper are shown in Table 2.

Table 2. Notation

Symbol	Description
$\mathbf{s}$	A sequence
n	The length of sequence $\mathbf{s}$
SDB	A sequence database consisting of N sequences
N	The number of sequences in SDB
$\mathbf{p}$	A pattern
m	The length of pattern $\mathbf{p}$
$sup(\mathbf{p}, \mathbf{s})$	The number of nonoverlapping occurrences of $\mathbf{p}$ in $\mathbf{s}$
$sup(\mathbf{p}, SDB)$	The number of nonoverlapping occurrences of $\mathbf{p}$ in SDB
Σ	The set of all characters
Ω, Γ	The sets of weak and strong characters, respectively
$minsup$	The minimum support threshold
a, b	The minimum and maximum wildcards, respectively
$wgap = (a, b)$	The weak-gap constraint

4. Algorithm design

There are two essential tasks in NWP mining: support calculation and candidate pattern generation. Sec-

tion 4.1 briefly reviews NETGAP as a method of calculating the support [45]. However, NETGAP is relatively inefficient, and Section 4.2 therefore presents our NetWeak algorithm, which employs depth-first and backtracking strategies to calculate the support more efficiently. Section 4.3 discusses the use of a pattern join strategy to generate candidate patterns, and Section 4.4 proposes the NWP-Miner algorithm and analyzes its time and space complexities.

4.1. NETGAP

NETGAP [45] employs a Nettore structure [44] to calculate the support. A Nettore that can represent all occurrences is an extended tree structure with multiple roots and parents. Some nodes in a Nettore have the same node ID but at different levels (for example, node n_4^8 represents a node at the fourth level with ID 8) [44]. **A path from a root to a leaf within the Nettore corresponds to an occurrence of a pattern in a sequence.** These properties mean that a Nettore is suitable for use in calculating the support. The NETGAP algorithm creates a Nettore based on a given pattern and sequence, and iteratively finds nonoverlapping occurrences and prunes the invalid nodes. Example 2 illustrates the principle of NETGAP.

Example 2. Suppose we have a sequence $s = \text{CaaOeACCaaefCaaBbeC}$, a pattern $p = \text{CaeC}$ with gap $[0,2]$. The corresponding Nettore is shown in Fig. 2.

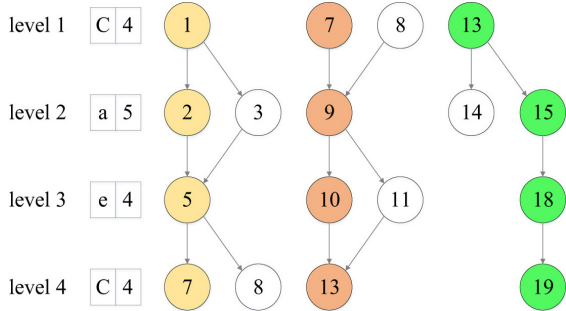


Fig. 2. Nettore for pattern p in sequence s created by NETGAP. The Nettore has four levels, since the pattern length is four. It also has four roots, n_1^1 , n_1^7 , n_1^8 and n_1^{13} , and four leaves, n_4^7 , n_4^8 , n_4^{13} and n_4^{19} . Some nodes have the same node ID. For example, both n_1^7 and n_4^7 have the same ID of 7. Nodes may have more than one parent. For example, node n_2^9 has two parents, n_1^7 and n_1^8 .

NETGAP [45] creates a Nettore based on the pattern p and sequence s . To avoid missing possible occurrences, NETGAP first finds and prunes invalid nodes that cannot reach a fourth-level leaf. In our example,

node n_2^{14} is pruned. NETGAP then obtains the left-most root-leaf path, and the corresponding occurrence is $\langle 1,2,5,7 \rangle$. The algorithm finds and prunes the invalid nodes n_2^3 and n_4^8 , and then iteratively obtains the left-most root-leaf path and prunes the invalid nodes along this path. In this way, occurrences $\langle 7,9,10,13 \rangle$ and $\langle 13,15,18,19 \rangle$ are found. There are therefore three occurrences in this example, i.e. the support is three.

NETGAP can also be used to calculate the support of a pattern with a weak-gap. Example 3 is used to illustrate this principle.

Example 3. We use the same sequence and pattern as in Example 2. We set $wgap = (0,2)$ and $\Omega = \{O, o, A, a, B, b, C, c\}$. The Nettore with weak-gap is shown in Fig. 3.

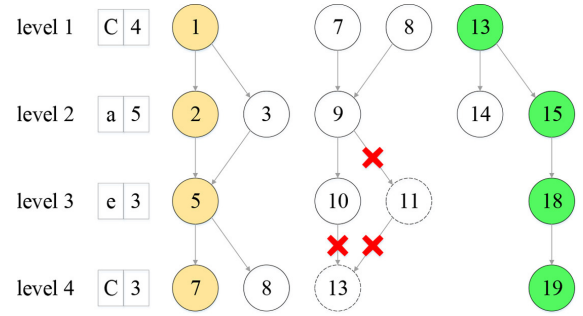


Fig. 3. Nettore for pattern p with weak-gap in sequence s created by NETGAP. Although $s_9 = p_2 = "a"$ and $s_{11} = p_3 = "e"$, node n_3^{11} cannot be created, since s_{10} is "e", a strong character that cannot be matched by the weak-gap $a(0,2)e$. Similarly, node n_4^{13} cannot be created since s_{11} is "e".

NETGAP finds two occurrences, $\langle 1,2,5,7 \rangle$ and $\langle 13,15,18,19 \rangle$, in Fig. 3, i.e. the support is two. Obviously, NETGAP can also be employed to calculate the support of pattern p with a weak-gap in sequence s . However, this approach is not efficient, and in the next section, we will present a more efficient algorithm called NetWeak.

4.2. NetWeak

From Examples 2 and 3, we know that NETGAP [45] has two drawbacks:

- (1) To find all nonoverlapping occurrences, NETGAP creates a whole Nettore.
- (2) It iteratively finds and prunes the invalid nodes.

To overcome these disadvantages, we propose NetWeak, which employs depth-first and backtracking strategies to find nonoverlapping occurrences with no

need to create a whole Nettoree. Suppose we have a sequence $\mathbf{s}$ and pattern $\mathbf{p}$ with weak-gap (a, b) . NetWeak performs the following steps.

Step 1: Find a root. For each character $s_i (1 \leq i \leq n)$ in sequence $\mathbf{s}$, if $s_i = p_1$, NetWeak creates a root n_1^i and selects node n_1^i as the current node.

Step 2: Find a child of the current node using depth-first and backtracking strategies. Suppose the current node is n_{j-1}^t and the last node at the j -th level is n_j^k . For each $s_i (t + a + 1 \leq i \leq t + b + 1 \text{ and } k + 1 \leq i)$, NetWeak checks whether or not $s_i = p_j$. There are four possible cases:

Case 1: If $s_i = p_j$, NetWeak creates node n_j^i and selects it as the current node.

Case 2: If s_i is a strong character, NetWeak selects the parent node of n_{j-1}^t as the current node.

Case 3: If $s_i \neq p_j$, NetWeak checks the next characters.

Case 4: If all of the characters satisfying the gap constraint are different from p_j , NetWeak selects the parent node of n_{j-1}^t as the current node.

Step 3: Iterate Step 2 until $j = m + 1$ or $j = 1$. If $j = m + 1$, then the path from n_1^i to n_m^t is a nonoverlapping occurrence.

Step 4: Iterate Step 1 until all characters in the sequence have been processed.

Example 4 illustrates the principle of NetWeak.

Example 4. We use the same pattern, sequence and weak-gap as in Example 3.

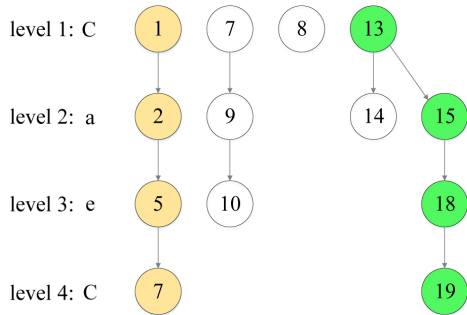


Fig. 4. Nettoree of pattern $\mathbf{p}$ in sequence $\mathbf{s}$ created by NetWeak. The Nettoree is simplified to a forest, since each node has at most one parent.

We select s_7 as example.

Step 1: Find a root. Since $s_7 = p_1 = C$, the root n_1^7 is created and selected as the current node.

Step 2: Find a child of the current node. Since $s_9 = p_2 = a$ and $9 - 7 - 1 = 1$, which satisfies the weak-gap constraint $C(0,2)a$, node n_2^9 is created and selected as the current node.

Algorithm 1 NetWeak

Input: Pattern $\mathbf{p}$, sequence $\mathbf{s}$, weak-gap constraint $wgap$, and weak character set Ω

Output: number of occurrences sup

```

1:  $sup \leftarrow 0$ ;
2: for each character  $s_i$  in sequence  $\mathbf{s}$  do
3:   if  $s_i = p_1$  then
4:     Create root  $n_1^i$ ;
5:      $j \leftarrow 2, t \leftarrow i$ ;
6:     while  $1 < j < m + 1$  do
7:       Find a child of the current node  $n_{j-1}^t$  using
       depth-first and backtracking strategies;
8:     end while
9:     if  $j = m + 1$  then
10:       $sup++$ ;
11:    end if
12:  end if
13: end for
14: return  $sup$ ;

```

Step 3: Iterate Step 2. NetWeak creates node n_3^{10} . s_{11} is “e”, and cannot be matched by the weak-gap constraint $e(0,2)c$ since “e” is a strong character. NetWeak therefore selects node n_2^9 as the current node and finds its other child. Node n_3^{11} cannot be created since s_{10} cannot be matched by the weak-gap constraint $a(0,2)e$. NetWeak therefore selects node n_1^7 as the current node. Similarly, node n_1^7 does not have a new child.

Step 4. Iterate Step 1. NetWeak finds a new root n_1^8 which does not have a child since node n_2^9 is a child of node n_1^7 and NetWeak does not allow a child to have multiple parents. Hence, NetWeak finds a new root n_1^{13} and obtains the occurrence $\langle 13, 15, 18, 19 \rangle$.

Pseudocode for NetWeak is given in Algorithm 1.

Theorem 1. The time and space complexities of NetWeak are both $O(m \times n)$, where m and n represent the pattern and sequence lengths, respectively.

PROOF. Obviously, the depth of the Nettoree is m levels. Each level has no more than n nodes, and each node can be visited no more than once. Since each node has only one parent, the complexity is independent of the size of the gap constraint. Hence, the time and space complexities of NetWeak for a sequence are both $O(m \times n)$.

The space and time complexities of NETGAP [45] are $O(m \times n \times w)$ and $O(m \times m \times n \times w)$, respectively, and NetWeak therefore outperforms NETGAP.

4.3. Candidate pattern generation

In this section, we employ a pattern join strategy to generate candidate patterns.

Definition 8. (Prefix and Suffix Patterns and Pattern Join) Suppose we have a pattern $\mathbf{p}$ with length m and event items c and d . If there is a pattern $\mathbf{q} = \mathbf{p}d$, then $\mathbf{p}$ is called the prefix pattern of $\mathbf{q}$, and is denoted by $\text{prefix}(\mathbf{q}) = \mathbf{p}$. If there is a pattern $\mathbf{r} = c\mathbf{p}$, then $\mathbf{p}$ is called the suffix pattern of $\mathbf{r}$, and is denoted by $\text{suffix}(\mathbf{r}) = \mathbf{p}$. Patterns $\mathbf{r}$ and $\mathbf{q}$ are super-patterns of pattern $\mathbf{p}$. Since $\text{suffix}(\mathbf{r}) = \text{prefix}(\mathbf{q}) = \mathbf{p}$, operator $\oplus$ is used to connect $\mathbf{r}$ and $\mathbf{q}$ to generate a super-pattern $\mathbf{h}$ with length $m + 2$, i.e. $\mathbf{h} = \mathbf{r} \oplus \mathbf{q} = c\mathbf{p}d$. This process is called pattern join.

Example 5. Suppose we have two patterns $\mathbf{p}_1 = \text{DB}$ and $\mathbf{p}_2 = \text{BC}$. We therefore obtain a super-pattern $\mathbf{h} = \mathbf{p}_1 \oplus \mathbf{p}_2 = \text{DBC}$.

In the NWP mining process, candidate patterns with different lengths need to be generated in a certain order, and the support is calculated separately. The traditional candidate pattern generation method uses a set enumeration tree strategy. Example 6 shows that our pattern join strategy outperforms the traditional approach.

Example 6. In Example 4, we have a sequence $\mathbf{s} = \text{CaaOeACCaeFCaaBbeC}$, $\text{wgap} = (0, 2)$, $\Sigma = \{\text{O}, \text{A}, \text{B}, \text{C}, \text{F}, \text{a}, \text{b}, \text{e}\}$, $\Omega = \{\text{O}, \text{o}, \text{A}, \text{a}, \text{B}, \text{b}, \text{C}, \text{c}\}$ and $\text{minsup} = 2$. There are four frequent NWPs with length three: $\{\text{"Caa"}, \text{"Cae"}, \text{"aae"}, \text{"aeC"}\}$.

The principle of the set enumeration tree strategy is that it adds each character in Σ to the end of each pattern to generate candidate patterns. Taking the frequent pattern "Caa" as an example, the set enumeration tree strategy generates eight candidate patterns $\{\text{"CaaO"}, \text{"CaaA"}, \text{"CaaB"}, \text{"CaaC"}, \text{"CaaF"}, \text{"Caaa"}, \text{"Caab"}, \text{"Caae"}\}$. It therefore generates $4 \times 8 = 32$ candidate patterns with length four. However, the pattern join strategy generates three candidate patterns $\{\text{"Caae"}, \text{"CaeC"}, \text{"aaeC"}\}$. Hence, the pattern join strategy outperforms the set enumeration tree strategy.

Table 3 gives a comparison of the number of candidate patterns.

4.4. NWP-Miner algorithm

In this section, we present our NWP-Miner algorithm and analyze its time and space complexities. The framework of NWP-Miner is shown in Fig. 5

NWP-Miner performs four steps.

Step 1: Scan the sequence database (*SDB*), calculate the support of each character, and store NWPs with length one into $\mathcal{P}$.

Step 2: Generate a candidate pattern set with length $m + 1$ based on NWPs with length m in $\mathcal{P}$, using the pattern join strategy.

Step 3: Calculate the support of a candidate pattern using NetWeak. If its support is no less than a given threshold, then store it in set $\mathcal{P}$.

Step 4: Iterate Steps 2 and 3 until the candidate pattern set is empty.

Example 7. We use Example 6 to illustrate the principle of NWP-Miner.

Step 1: After scanning the sequence, NWP-Miner discovers NWPs with length one, i.e. $\mathcal{P}_1 = \{\text{"C"}, \text{"a"}, \text{"e"}\}$.

Step 2: NWP-Miner obtains $\mathcal{C}_2 = \{\text{"CC"}, \text{"Ca"}, \text{"Ce"}, \text{"aC"}, \text{"aa"}, \text{"ae"}, \text{"eC"}, \text{"ea"}, \text{"ee"}\}$ based on $\mathcal{P}_1$.

Step 3: For each pattern in $\mathcal{C}_2$, the support is calculated. For example, if $\mathbf{p} = \text{Ca}$, then after calculating the support of pattern $\mathbf{p}$ we know that $\text{sup}(\mathbf{p}, \mathbf{s}) = 3 \geq 2$. Thus, pattern $\mathbf{p}$ is an NWP and is stored in set $\mathcal{P}_2$. Hence $\mathcal{P}_2 = \{\text{"Ca"}, \text{"aa"}, \text{"ae"}, \text{"eC"}\}$.

Step 4: Iterate Steps 2 and 3. NWP-Miner discovers $\mathcal{P}_3 = \{\text{"Caa"}, \text{"Cae"}, \text{"aae"}, \text{"aeC"}\}$, $\mathcal{P}_4 = \{\text{"Caae"}, \text{"CaeC"}, \text{"aaeC"}\}$ and $\mathcal{P}_5 = \{\text{"CaaeC"}\}$. Since the NWP set $\mathcal{P}_5$ cannot generate the candidate pattern set $\mathcal{C}_6$, NWP mining is finished.

Pseudocode for NWP-Miner is given in Algorithm 2.

Theorem 2. The time and space complexities of NWP-Miner are $O(M \times T \times l)$ and $O(m \times n)$, respectively, where M , T , and l are the max length of candidate patterns, the total length of *SDB*, and the number of candidate patterns, respectively.

PROOF. The time complexity of NWP-Miner contains two parts: support calculation and candidate pattern generation. According to Theorem 1, the time complexity of NetWeak is $O(m \times n)$. Therefore, it is easy to know that the time complexity of calculating candidate pattern support in sequence database is $O(M \times T)$. Since there are l candidate patterns, the time complexity of calculating all candidate patterns is $O(M \times T \times l)$. Now, we analyze the time complexity of candidate pattern generation. Suppose there are c candidate patterns with length m . Thus, the time complexity of PatternJoin to generate candidate patterns with $m + 1$ is $O(c \times c)$. Therefore, the time complexity of generating all candidate patterns is $O(M \times c \times c)$. Generally, c is much less

Table 3. Comparison of number of candidate patterns

Pattern length	Length = 1	Length = 2	Length = 3	Length = 4	Length = 5	Length = 6	Total
Number of candidate patterns by enumeration tree	8	24	32	32	24	8	128
Number of candidate patterns by pattern join	8	9	5	3	1	0	26
Number of NWP	3	4	4	3	1	0	15

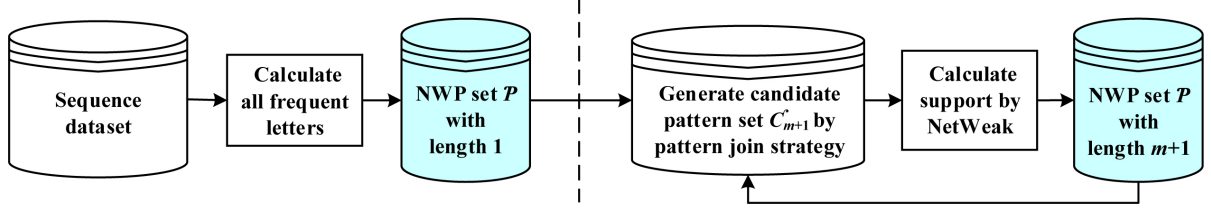


Fig. 5. Framework of NWP-Miner

than T , and c is also less than l . Hence, the time complexity of NWP-Miner is $O(M \times T \times l + M \times c \times c) = O(M \times T \times l)$.

The space complexity of NWP-Miner contains three parts: calculating support and storing candidate patterns and frequent NWP. NetWeak releases the space after calculating support for a pattern in a sequence. Therefore, no matter how many times NetWeak runs, the space complexity of calculating support is the same as that of NetWeak, i.e., the space complexity of calculating support is $O(M \times n)$. Since there are l candidate patterns and the maximum candidate length is M , the space complexity of storing candidate patterns is $O(M \times l)$. Since the number of NWP is less than that of candidate patterns, the space complexity of storing frequent NWP is also $O(M \times l)$. Generally, l is less than n . Hence, the space complexity of NWP-Miner is $O(M \times n + M \times l + M \times l) = O(M \times n)$.

5. Experimental analysis

Section 5.1 presents a pre-processing method of converting time series data into character sequences and benchmark datasets. Section 5.2 introduces the competitive algorithms considered here. Since NWP-Miner involves two key steps, support calculation and candidate pattern generation, Section 5.3 validates the efficiency of the two key steps. Section 5.4 employs some state-of-the-art methods to further evaluate the mining performance. Section 5.5 presents a case study based on data from the Dow Jones stock index and West Texas Intermediate.

All experiments are carried out on a computer with an Intel(R)Core(TM)i7-8550U processor, 8 GB memory, Windows 10 operating system, and VC++6.0 as the

experimental environment. Algorithms and data can be downloaded from <https://github.com/wuc567/Pattern-Mining/tree/master/NWP-Miner>.

5.1. Data preprocessing method and benchmark datasets

To evaluate the proposed NWP-Miner algorithm in a more intuitive way, experiments are carried out on a set of time series data. Since a time series is a series of continuous numerical data, it is first converted into character sequences.

In this section, self-adaptive data pre-processing method is proposed based on the weak character rate ρ , where ρ is the ratio of the total number of weak characters to the total number of characters.

Step 1. Calculate the gradient according to the following equation:

$$f_i = (t_{i+1} - t_i)/t_i, \quad (1)$$

where $1 \leq i \leq n - 1$.

Step 2. According to ρ , generate a conversion table based on f_i . It is worth noting that users can set the weak character rate ρ and the character set of the sequence by themselves according to the needs in different fields.

For example, in stock, users generally pay more attention to the large fluctuations that do not occur frequently. We set the strong character rate is 10%. Therefore, the weak character rate is $\rho=90\%$ for the Shanghai Composite Index stock dataset. Moreover, we set the character set of the sequence $\Sigma = \{O, A, B, C, D, E, F, G, H, o, a, b, c, d, e, f, g, h\}$ and the weak character set is $\Sigma = \{O, o, A, a, B, b, C, c\}$. If the gradient is a positive number, it is represented by an upper case, otherwise, it is represented by a lower case. Since the gradient of

Algorithm 2 NWP-Miner

Input: Sequence database SDB , support threshold $minsup$, weak-gap constraint $wgap$ and weak character set Ω

Output: NWPs $\mathcal{P}$

```
1: Traverse  $SDB$  and store all frequent letters in set  $\mathcal{P}_1$ ;
2:  $\mathcal{P} \leftarrow \mathcal{P}_1$ ;
3:  $C_2 \leftarrow \text{PatternJoin}(\mathcal{P}_1)$ ;
4:  $m \leftarrow 2$ ;
5: while  $C_m \neq \text{NULL}$  do
6:   for each  $\mathbf{p}$  in  $C_m$  do
7:      $sup \leftarrow 0$ ;
8:     for each sequence  $\mathbf{s}$  in  $SDB$  do
9:        $sup \leftarrow sup + \text{NetWeak}(\mathbf{p}, \mathbf{s})$ ;
10:    if  $sup \geq minsup$  then
11:       $\mathcal{P}_m \leftarrow \mathcal{P}_m \cup \mathbf{p}$ ;
12:      break;
13:    end if
14:  end for
15: end for
16:  $\mathcal{P} \leftarrow \mathcal{P} \cup \mathcal{P}_m$ ;
17:  $C_{m+1} \leftarrow \text{PatternJoin}(\mathcal{P}_m)$ ;
18:  $m \leftarrow m + 1$ ;
19: end while
20: return  $\mathcal{P}$ ;
```

90% is between -3% and 3%, the range (0, 3%] is represented by ‘O’, ‘A’, ‘B’, and ‘C’. Since the gradient obeys the normal distribution, to make the appearance probabilities of ‘O’, ‘A’, ‘B’, and ‘C’ equal, the sizes of intervals are different. Similarly, the sizes of intervals of ‘D’, ‘E’, ‘F’, ‘G’, and ‘H’ are also different. The code table is generated as shown in Table 4. **Hence, a time series dataset can be converted into character sequences.** Table 5 shows the benchmark datasets used here.

5.2. Baseline method

To evaluate the performance of NWP-Miner, seven competitive algorithms are selected: NWP-bf, NWP-df, NETGAP-NWP, NWP-nettree, GSgrow, NOSEP and NWP-noweak. The principles of these algorithms can be summarized as follows:

1. **NWP-bf and NWP-df:** To determine the efficiency of the pattern join strategy used in NWP-Miner to generate candidate patterns, we create the NWP-bf and NWP-df algorithms, which use breadth-first and depth-first set enumeration tree strategies to generate candidate patterns, respectively.
2. **NETGAP-NWP:** To validate the performance of NetWeak, NETGAP-NWP uses NETGAP [45] to

Algorithm 3 PatternJoin

Input: NWPs with length m , i.e. $\mathcal{P}_m$

Output: Candidate patterns with length $m+1$, i.e. C_{m+1}

```
1:  $C_{m+1} \leftarrow \{\}$ ;
2: for each pattern  $\mathbf{p}$  in  $\mathcal{P}_m$  do
3:   for each pattern  $\mathbf{q}$  in  $\mathcal{P}_m$  do
4:     if  $\text{suffix}(\mathbf{p}) == \text{prefix}(\mathbf{q})$  then
5:        $\mathbf{r} \leftarrow \mathbf{p} \oplus \mathbf{q}$ ;
6:        $C_{m+1} \leftarrow C_{m+1} \cup \mathbf{r}$ ;
7:     end if
8:   end for
9: end for
10: return  $C_{m+1}$ ;
```

calculate the support. To completely calculate the support, NETGAP creates a whole Nettree and has to prune the invalid nodes after finding an occurrence.

3. **NWP-nettree:** To further validate the performance of depth-first and backtracking strategies, we propose the NWP-nettree algorithm, which adopts the two strategies to create part of a Nettree. Therefore, many useless nodes will not be created. Then, NWP-nettree applies the two strategies to calculate the support.
4. **GSgrow [3] and NOSEP [45]:** To further evaluate the performance of NWP-Miner, we also consider two state-of-the-art algorithms, GSgrow and NOSEP, which mine frequent nonoverlapping patterns.
5. **NWP-noweak:** To explore the influence of the weak-gap constraints, NWP-noweak mines frequent patterns using traditional gap constraints which are the same as those of NOSEP.

5.3. Efficiency of pattern join strategy and NetWeak algorithm

We know that NWP-Miner employs the pattern join strategy to generate candidate patterns and the NetWeak algorithm to calculate the support. To evaluate the efficiency of the strategies of the two key steps, four competitive algorithms are selected: NWP-bf, NWP-df, NETGAP-NWP, and NWP-nettree. We use six databases to carry out the experiments: S1, S2, S4, S5, S7, and S8. The parameters are $wgap = (0, 3)$, $minsup = 100$ and $\Omega = \{O, o, A, a, B, b, C, c\}$. Since all the five algorithms are complete, the number of NWPs is the same for all methods. There are 48, 120, 183, 73, 282, and 84 NWPs on S1, S2, S4, S5, S7, and S8, respectively. The comparisons of running time, number

Table 4. Code table

f_i	Code	f_i	Code
(0%, 0.35%]	O	(-0.35%, 0%]	o
(0.35%, 0.85%]	A	(-0.85%, -0.35%]	a
(0.85%, 1.45%]	B	(-1.45%, -0.85%]	b
(1.45%, 3%]	C	(-3%, -1.45%]	c
(3%, 3.5%]	D	(-3.5%, -3%]	d
(3.5%, 4%]	E	(-4%, -3.5%]	e
(4%, 5%]	F	(-5%, -4%]	f
(5%, 7%]	G	(-7%, -5%]	g
(7%, Inf%]	H	(-Inf%, -7%]	h

Table 5. Benchmark datasets

Dataset	Type	From	Dates	Number of sequences	Length
S1	Stock	Dow Jones Industrial Average	2007/1/4–2020/2/26	1	3,307
S2	Stock	S&P 500 Index	2000/8/14–2020/2/26	1	4,998
S3	Stock	Shanghai Composite Index	1990/12/19–2020/2/26	1	7,133
S4	Stock	Shenzhen Component Index	1991/4/3–2020/2/26	1	7,043
S5	Oil	World Trade Index	2000/7/26–2020/2/26	1	4,995
S6	Oil	Brent Crude Index	2000/9/4–2020/2/26	1	4,999
S7	Weather	Beijing Tiantan AQI	2019/1/1–2020/2/22	1	8,404
S8	Music	Bach Chorales	–	100	4,593
S9	Stock	Dow Jones Index	2019/5/1–2020/2/13	1	200
S10	Crude Oil	West Texas Intermediate (WTI)	2019/1/1–2019/10/2	1	200

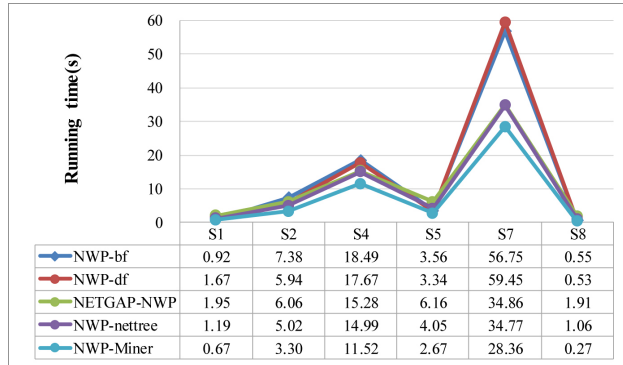


Fig. 6. Comparison of running time with competitive algorithms

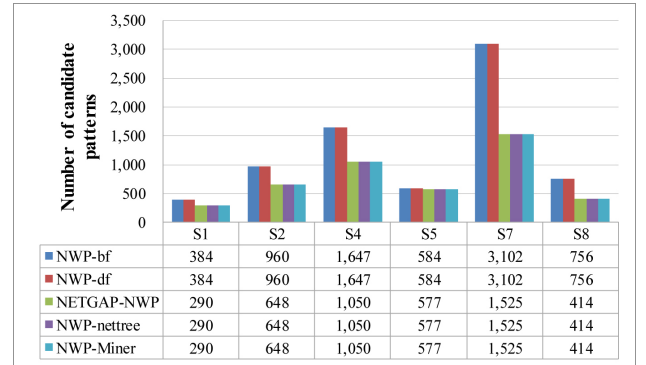


Fig. 7. Comparison of number of candidate patterns with competitive algorithms

of candidate patterns, and number of nodes created are shown in Figs. 6 – 9, respectively.

The results indicate the following observations:

1. The pattern join strategy is more efficient than the breadth-first and depth-first set enumeration tree strategies in candidate pattern generation. We know that NWP-Miner employs the pattern join strategy to generate candidate patterns, while NWP-bf and NWP-df adopt the breadth-first and depth-first strategies, respectively. Fig. 6 shows that NWP-Miner is faster than both NWP-bf and NWP-df. For example, the runtime of NWP-Miner is 28.36 s on S7, while the runtime of NWP-bf and NWP-df is 56.75 and 59.45 s, respectively. The reason is that

NWP-Miner only calculates 1,525 candidate patterns, while NWP-bf and NWP-df calculate 3,012 candidate patterns, as shown in Fig. 7. The pattern join strategy is therefore more efficient than the other two approaches, which is consistent with the analysis in Section 4.3. Moreover, memory usage of the pattern join strategy is almost the same as those of the breadth-first and depth-first set enumeration tree strategies. Fig. 9 reports that the memory usage of NWP-Miner, NWP-bf, and NWP-df is 0.3, 0.4, and 0.3 M on S1, respectively. The same phenomenon can be found on all other datasets. The reason is as follows. We know that NWP-Miner, NWP-bf, and NWP-df use different strategies to generate candi-

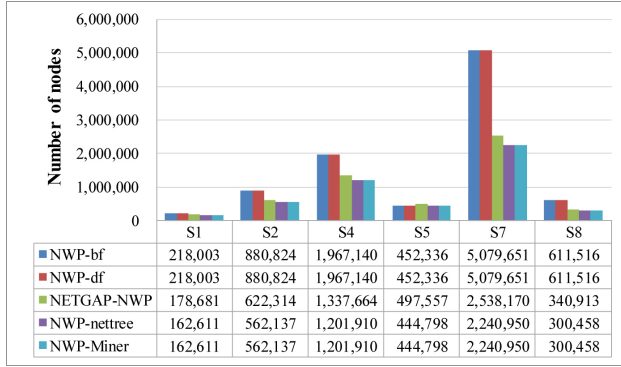


Fig. 8. Comparison of number of nodes with competitive algorithms

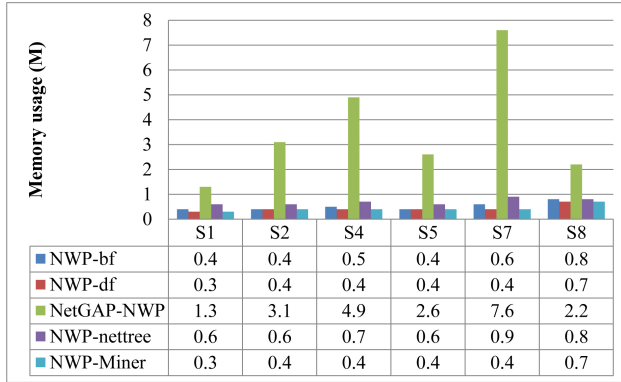


Fig. 9. Comparison of memory usage with competitive algorithms

date patterns. As the analysis of the space complexity of NWP-Miner, compared with the support calculation, the space of candidate pattern generation can be neglected. Therefore, the memory usage of the three algorithms is almost the same. Hence, NWP-Miner outperforms NWP-bf and NWP-df.

2. The depth-first and backtracking strategies are efficient strategies in support calculation. We know that NWP-Miner and NETGAP-NWP discover the same number of NWP, and Figs. 6 – 7 show that NWP-Miner and NETGAP-NWP calculate the same number of candidate patterns, but NWP-Miner runs faster than NETGAP-NWP. For example, the running time of NETGAP-NWP and NWP-Miner is 15.28 and 11.52 s on S4, respectively. The reason is that NWP-Miner employs the depth-first and backtracking strategies to calculate the support, and reduces the running time by pruning invalid nodes. The number of nodes created by NETGAP-NWP and NWP-Miner is 1,337,664 and 1,201,910 on S4, respectively. Moreover, the time complexity is reduced from $O(m \times m \times n \times W)$ to $O(m \times n)$. More importantly,

Fig. 9 reports that NWP-Miner consumes less memory than NETGAP-NWP. For example, the memory usage of NWP-Miner is 0.4 M on S4, while that of NETGAP-NWP is 4.9 M. The reason is that NWP-Miner does not need to create a whole Nettree to calculate the support, while NETGAP-NWP has to create a whole Nettree which contains many useless nodes and parent-child relationships. Thus, NWP-Miner not only runs faster than NETGAP-NWP, but also consumes less memory. Hence, NWP-Miner outperforms NETGAP-NWP.

3. NWP-Miner gives better performance than NWP-nettree. We know that NWP-Miner and NWP-nettree find the same number of NWP, and Figs. 6 – 7 show that NWP-Miner and NWP-nettree calculate the same number of candidate patterns and created nodes, but NWP-Miner runs faster than NWP-nettree. For example, in Fig. 6, the running time of NWP-nettree and NWP-Miner is 5.02 and 3.30 s on S2, respectively. The reason for this phenomenon is as follows. NWP-nettree initially creates part of a Nettree, in which some nodes can have multiple parents. However, NWP-Miner creates a simplified Nettree, and all nodes have only one parent, meaning that it avoids the generation of redundant parent relationships. Fig. 9 also validates that NWP-Miner consumes less memory than NWP-nettree. For example, the memory usage of NWP-Miner is 0.4 M on S2, while that of NWP-nettree is 0.6 M. Thus, NWP-Miner not only runs faster than NWP-nettree, but also consumes less memory. Hence, NWP-Miner outperforms NWP-nettree.

In conclusion, NWP miner is equipped with two effective strategies to generate candidate patterns and calculate the support.

5.4. Mining performance

To verify the mining performance of NWP-Miner, we select databases S1, S3, S4, S6, S7, and S8 to carry out experiments with $minsup = 150$, $gap = [0, 3]$ and $\Omega = \{O, o, A, a, B, b, C, c\}$. Three competitive algorithms are selected: GSgrow [3], NOSEP [45], and NWP-noweak. Comparison of the number of NWP, running time, number of candidate patterns, and memory usage are shown in Figs. 10 – 13, respectively.

The results indicate that NWP-Miner outperforms GSgrow, NOSEP and NWP-noweak.

1. NWP-Miner discovers fewer patterns than the other three algorithms. For example, it can be seen from Fig. 10 that NWP-Miner finds 18 patterns on S1,

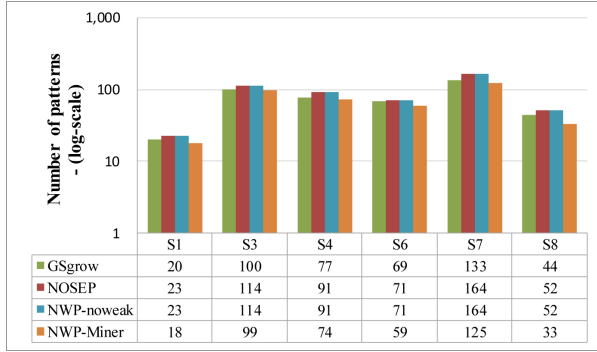


Fig. 10. Comparison of number of patterns

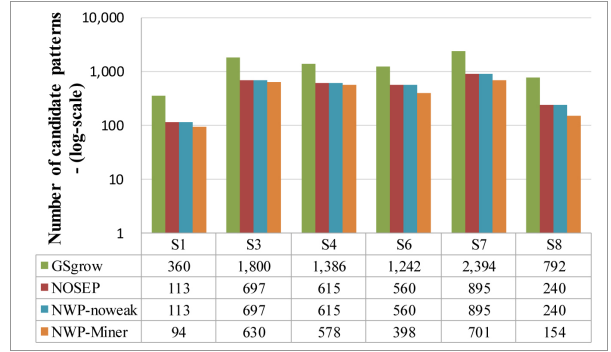


Fig. 12. Comparison of number of candidate patterns

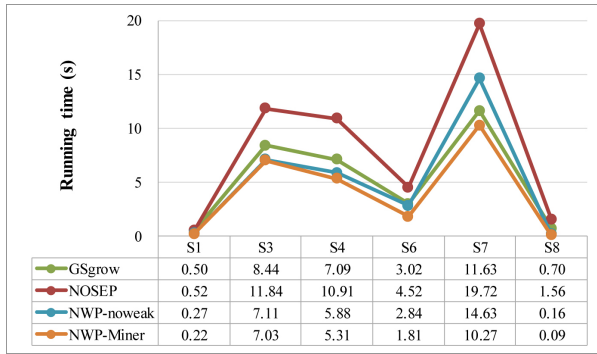


Fig. 11. Comparison of running time

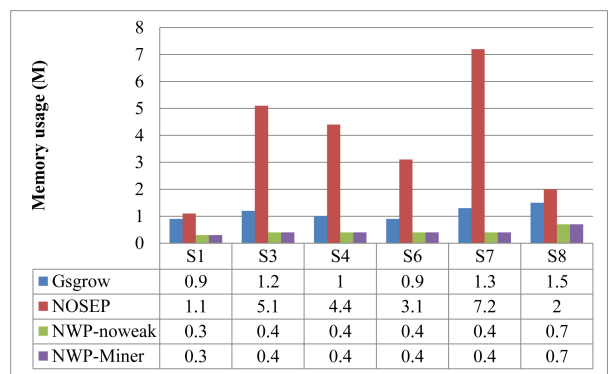


Fig. 13. Comparison of memory usage

- while GSgrow, NOSEP and NWP-noweak find 20, 23 and 23, respectively. As discussed in Fig. 1, we know that although Fig. 1 (a) is significantly different from Fig. 1 (c), Fig. 1 (c) can be seen as an occurrence of Fig. 1 (a) according to the existing approaches. NWP-Miner can effectively avoid this phenomenon using weak-gaps. GSgrow, NOSEP, and NWP-noweak discover frequent patterns based on traditional gaps, and therefore find more patterns, but these extra patterns are noise patterns.
2. NWP-Miner runs faster and checks fewer candidate patterns than the other three algorithms. For example, on S1, the runtime of NWP-Miner is 0.22 s, while that of the other three algorithms is 0.50, 0.52 and 0.27 s, respectively. NWP-Miner checks 94 candidate patterns, while the three alternatives check 360, 113 and 113, respectively. The reason for this is as follows. Since some occurrences with significant deviations can be found, the support with the weak-gap constraint is less than that with traditional gaps. Compared with NWP-noweak, NWP-Miner reduces the number of candidate patterns, and therefore runs faster. As shown in the analysis in Section 4.2, NetWeak adopts the depth-first and backtrack-

ing strategies to calculate the pattern support, which reduces the time complexity from $O(m \times m \times n \times W)$ to $O(m \times n)$. Hence, compared with GSgrow and NOSEP, NWP-Miner adopts a more efficient strategy to calculate the support and employs weak-gaps to reduce the candidate patterns, meaning that it runs faster.

3. NWP-Miner consumes less memory than GSgrow and NOSEP, the same memory as NWP-noweak. Fig. 13 reports that on S3, the memory usage of GSgrow, NOSEP, and NWP-noweak is 1.2, 5.1, and 0.4 M, respectively, while that of NWP-Miner is 0.4. The same phenomenon can be found on all other datasets. The reason is as follows. To calculate the supports of superpatterns, GSgrow stores the results of subpatterns, which consumes a lot of memory. NOSEP has to create a whole Nettee, which contains a lot of useless nodes and parent-child relationships. Therefore, NOSEP consumes more memory than NWP-Miner. As the analysis of space complexity of NWP-Miner, the support calculation is the main part of memory consumption. NWP-noweak and NWP-Miner adopt the same principle to calcu-

late the supports. Therefore, NWP-Miner consumes the same memory as NWP-noweak.

In conclusion, NWP-Miner can effectively filter out noise patterns with fast running speed and less memory.

5.5. Case study

In this section, we employ NOSEP [45] and NWP-Miner to mine the patterns on the S_9 and S_{10} datasets, which are the sequences of Dow Jones stock index and West Texas Intermediate, respectively. The parameters for NOSEP to mine frequent patterns (NFPs) on S_9 are $gap = [0, 3]$ and $minsup = 5$, and those for NWP-Miner on S_9 are $wgap = (0, 3)$, $minsup = 5$. Moreover, the parameters for NOSEP to mine NFPs on S_{10} are $gap = [0, 3]$ and $minsup = 4$, and those for NWP-Miner on S_{10} are $wgap = (0, 3)$, $minsup = 4$. $\Omega = \{O, o, A, a, B, b, C, c\}$. Cloud lexicons [13] are used to visualize the NFPs and NFPs shown in Fig. 14. In a cloud lexicon, the larger the font size is, the more frequent the pattern is.

S9-NFPs



(a) 90 NFPs in S_9

S9-NWPs



(b) 43 NWPs in S_9

S10-NFPs



(c) 152 NFPs in S_{10}

S10-NWPs



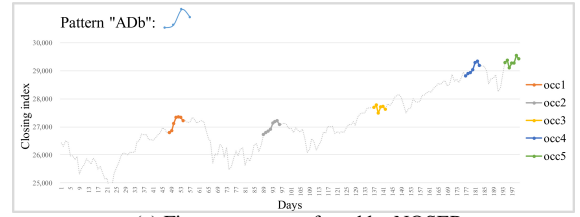
(d) 50 NWPs in S_{10}

Fig. 14. Comparison of mined NFPs and NWPs

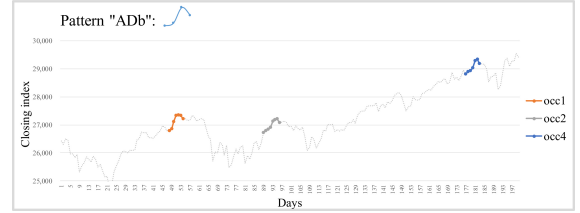
The results indicate the following observations.

NWP mining outperforms traditional Nonoverlapping SPM, since NWP-Miner can effectively filter out noise patterns. Thus, NWP-Miner discovers fewer patterns than NOSEP. For example, Fig. 14 shows that

on S_9 , NOSEP discovers 90 NFPs, while NWP-Miner mines 43 NWPs. The same phenomenon can be found on S_{10} . For example, in Fig. 14 (a), pattern “ADb” can be found by NOSEP, but not by NWP-Miner. Moreover, in Fig. 14 (c), pattern “Afo” can be found by NOSEP, but not by NWP-Miner. To further illustrate the reason, Figs. 15 and 16 show the comparison of occurrences of pattern “ADb” on S_9 and pattern “Afo” on S_{10} , respectively.

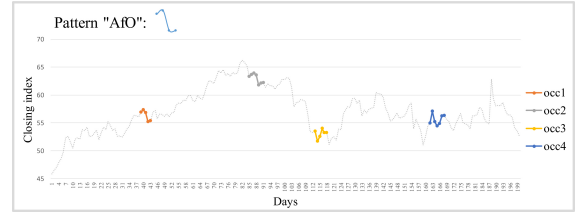


(a) Five occurrences found by NOSEP

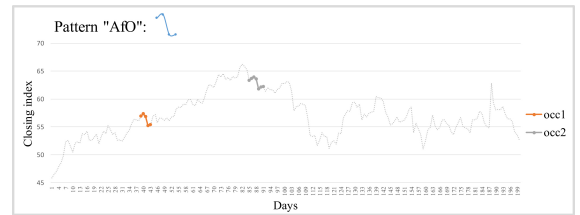


(b) Three occurrences found by NWP-Miner

Fig. 15. Comparison of matching results of pattern “AeO” on S_9



(a) Four occurrences found by NOSEP



(b) Two occurrences found by NWP-Miner

Fig. 16. Comparison of matching results of pattern “aBf” on S_{10}

Fig 15 shows that there are five occurrences found by NOSEP, while three occurrences by NWP-Miner. However, the trends of occ3 and occ5 are significantly different from pattern “ADb”. Since $minsup = 5$, NOSEP finds pattern “ADb”, while NWP-Miner does not. Fig.

16 also shows the same phenomenon, since occ3 and occ4 are significantly different from pattern “AfO”. Thus, the reliability of the mining results of NOSEP is poor, while that of NWP-Miner is good. Hence, NWP-Miner can effectively filter out noise patterns and is more accurate than NOSEP.

6. Conclusion

Gap constraint SPM is more flexible than other approaches, since it allows users to set a gap according to their specific needs. The patterns mined in this way may contain some noise, since the gap can match any character. To tackle this issue, we propose a method of NWP mining that uses a weak-gap to make it only match weak characters, which can therefore mine the required patterns more accurately. To mine these NWP, we propose the NWP-Miner algorithm, which adopts the Netweak algorithm to calculate the support and a pattern join strategy to generate candidate patterns. Netweak employs the depth-first search and backtracking strategies to find occurrences, and avoids generating redundant nodes and parent-child relationships. In this way, the time and space complexities can be improved. **Extensive experiments conducted on real stock prices and oil, weather and music datasets verify the effectiveness of NWP-Miner. The results also validate that the NWP-Miner algorithm runs faster and consumes less memory than the state-of-the-art algorithms, since it adopts the pattern join strategy to generate candidate patterns and the depth-first and backtracking strategies to calculate the support, which does not need to create a whole Ntree to calculate the support. Therefore, the creation of redundant nodes and parent-child relationships can be avoided. More importantly, the experimental results on Dow Jones stock data and West Texas Intermediate show that NWP-Miner can effectively filter out noise patterns and discover more meaningful patterns, since the trends of the frequent patterns mined by NWP-Miner are the same, while those of the noise patterns are different.**

Acknowledgement

This work was partly supported by National Natural Science Foundation of China (61976240, 52077056, 917446209), National Key Research and Development Program of China (2016YFB1000901), and Natural Science Foundation of Hebei Province, China (Nos. F2020202013, E2020202033).

References

- [1] R. Bai, W. K. Hon, E. Lo, Z. He, K. Zhu, Historic moments discovery in sequence data, *ACM Transactions on Database Systems* 44 (1) (2019) 3.
- [2] Y. Chen, X. Zhao, X. Lin, Y. Wang, D. Guo, Efficient mining of frequent patterns on uncertain graphs, *IEEE Transactions on Knowledge and Data Engineering* 31 (2) (2019) 287–300.
- [3] B. Ding, D. Lo, J. Han, S. C. Khoo, Efficient mining of closed repetitive gapped subsequences from a sequence database, in: 2009 IEEE International Conference on Data Engineering, 2009, 1024–1035.
- [4] X. Dong, Y. Gong, L. Cao, e-RNSP: An efficient method for mining repetition negative sequential patterns, *IEEE Transactions on Cybernetics* 50 (5) (2020) 2084–2096.
- [5] M. Fabrègue, A. Braud, S. Bringay, F. L. Ber, M. Teisseire, Mining closed partially ordered patterns, a new optimized algorithm, *Knowledge-Based Systems* 79 (2015) 68–79.
- [6] P. Fournier-Viger, C. Cheng, Z. Cheng, J. C. Lin, N. Selmaoui-Folcher, Mining significant trend sequences in dynamic attributed graphs, *Knowledge-Based Systems* 182 (2019) 104797.
- [7] P. Fournier-Viger, A. Gomariz, T. Gueniche, A. Soltani, C. Wu, V. S. Tseng, SPMF: A java open-source pattern mining library, *Journal of Machine Learning Research* 15 (1) (2014) 3389–3393.
- [8] P. Fournier-Viger, P. Yang, R. U. Kiran, S. Ventura, J. M. Luna, Mining local periodic patterns in a discrete sequence, *Information Sciences*, 544 (2021) 519–548.
- [9] W. Gan, J. C. Lin, P. Fournier-Viger, H. Chao, P. S. Yu, A survey of parallel sequential pattern mining, *ACM Transactions on Knowledge Discovery from Data* 13 (3) (2019) 25.
- [10] W. Gan, J. C. Lin, J. Zhang, H. Chao, H. Fujita, P. S. Yu, ProUM: Projection-based utility mining on sequence data, *Information Sciences* 513 (2020) 222–240.
- [11] D. Guo, X. Hu, F. Xie, X. Wu, Pattern matching with wildcards and gap-length constraints based on a centrality-degree graph, *Applied Intelligence* 39 (1) (2013) 57–74.
- [12] Z. He, S. Zhang, F. Gu, J. Wu, Mining conditional discriminative sequential patterns, *Information Sciences* 478 (2019) 524–539.
- [13] F. Heimerl, S. Lohmann, S. Lange, T. Ertl, Word Cloud Explorer: Text analytics based on word clouds, in: 2014 47th Hawaii International Conference on System Sciences, 2014, 1833–1842.
- [14] Y. Huang, X. Wu, X. Hu, F. Xie, J. Gao, G. WU, Mining frequent patterns with gaps and one-off condition, in: 2009 International Conference on Computational Science and Engineering, 2009, 180–186.
- [15] H. Jiang, X. Li, Z. Ren, J. Xuan, Z. Jin, Toward better summarizing bug reports with crowdsourcing elicited attribute, *IEEE Transactions on Reliability* 68 (1) (2019) 2–22.
- [16] H. T. Lam, F. Mörchén, D. Fradkin, T. Calders, Mining compressing sequential patterns, *Statistical Analysis and Data Mining* 7 (1) (2014) 34–52.
- [17] B. Le, M. Tran, B. Vo, Mining frequent closed inter-sequence patterns efficiently using dynamic bit vectors, *Applied Intelligence* 43 (1) (2015) 74–84.
- [18] C. Li, Q. Yang, J. Wang, M. Li, Efficient mining of gap-constrained subsequences and its various applications, *ACM Transactions on Knowledge Discovery from Data* 6 (1) (2012) 2.
- [19] Y. Li, L. Yu, J. Liu, L. Guo, Y. Wu, Xindong Wu, NetDPO: (delta, gamma)-approximate pattern matching with gap constraints under one-off condition, *Applied Intelligence* (2021) DOI: 10.1007/s10489-021-03000-2
- [20] Y. Li, S. Zhang, L. Guo, J. Liu, Y. Wu, X. Wu, NetNMSF:

- Nonoverlapping maximal sequential pattern mining. *Applied Intelligence* (2021) DOI: 10.1007/s10489-021-02912-3
- [21] J.C.W. Lin, T. Li, M. Pirouz, J. Zhang, P. Fournier-Viger, High average-utility sequential pattern mining based on uncertain databases, *Knowledge and Information Systems* 62 (6) (2020) 1199–1228.
 - [22] F. Min, Y. Wu, X. Wu, The Apriori property of sequence pattern mining with wildcard gaps, *International Journal of Functional Informatics and Personalised Medicine* 4 (1) (2012) 15–31.
 - [23] F. Min, Z. Zhang, W. Zhai, R. Shen, Frequent pattern discovery with tri-partition alphabets, *Information Sciences* 507 (2020) 715–732.
 - [24] H. Park, J. Y. Jung, SAX-ARM: Deviant event pattern discovery from multivariate time series using symbolic aggregate approximation and association rule mining, *Expert Systems with Applications* 141 (2020) 112950.
 - [25] P. Qiu, Y. Gong, Y. Zhao, L. Cao, C. Zhang, X. Dong, An efficient method for modeling nonoccurring behaviors by negative sequential patterns with loose constraint, *IEEE Transactions on Neural Networks and Learning Systems* (2021) doi:10.1109/TNNLS.2021.3063162.
 - [26] A. Renz-Wieland, M. Bertsch, R. Gemulla, Scalable frequent sequence mining with flexible subsequence constraints, in: 2019 IEEE 35th International Conference on Data Engineering, 2019, 1490–1501.
 - [27] W. Song, Y. Liu, J. Li, Mining high utility itemsets by dynamically pruning the tree structure, *Applied Intelligence* 40(1) (2014) 29–43.
 - [28] W. Song, L. Liu, C. Huang, Generalized maximal utility for mining high average-utility itemsets, *Knowledge and Information Systems* 63 (11) (2021) 2947–2967.
 - [29] Q. Shi, J. Shan, W. Yan, Y. Wu, X. Wu, NetNPG: Nonoverlapping pattern matching with general gap constraints, *Applied Intelligence* 50 (6) (2020) 1832–1845.
 - [30] S. Sumalatha, R. B. V. Subramanyam, Distributed mining of high utility time interval sequential patterns using mapreduce approach, *Expert Systems with Applications* 141 (2020) 112967.
 - [31] C. Tan, F. Min, M. Wang, H. Zhang, Z. Zhang, Discovering patterns with weak-wildcard gaps, *IEEE Access* 4 (2016) 4922–4932.
 - [32] T. Truong, H. Duong, B. Le, P. Fournier-Viger, U. Yun, Efficient high average-utility itemset mining using novel vertical weak upper-bounds, *Knowledge-Based Systems* 183 (2019) 104847.
 - [33] Y. Wang, Y. Wu, Y. Li, F. Yao, P. Fournier-Viger, X. Wu, Self-adaptive nonoverlapping sequential pattern mining. *Applied Intelligence* (2021) DOI: 10.1007/s10489-021-02763-y
 - [34] L. Wang, X. Bao, L. Zhou, Redundancy reduction for prevalent co-location patterns, *IEEE Transactions on Knowledge and Data Engineering* 30 (1) (2018), 142–155.
 - [35] T. Wang, L. Duan, G. Dong, Z. Bao, Efficient mining of outlying sequence patterns for analyzing outlieriness of sequence data, *ACM Transactions on Knowledge Discovery from Data* 14 (5) (2020) 62.
 - [36] M. K. Warmuth, D. Haussler, On the complexity of iterated shuffle, *Journal of Computer and System Sciences* 28 (3) (1984) 345–358.
 - [37] M. Wu, X. Wu, On big wisdom, *Knowledge and Information Systems* 58 (1) (2019) 1–8.
 - [38] X. Wu, X. Zhu, Y. He, A. N. Arslan, PMBC: Pattern mining from biological sequences with wildcard constraints, *Computers in Biology and Medicine* 43 (5) (2013) 481–492.
 - [39] Y. Wu, J. Fan, Y. Li, L. Guo, X. Wu, NetDAP: (delta, gamma) approximate pattern matching with length constraints, *Applied Intelligence* 50 (11) (2020) 4094–4116.
 - [40] Y. Wu, M. Geng, Y. Li, L. Guo, Z. Li, P. Fournier-Viger, X. Zhu, X. Wu, HANP-Miner: High average utility nonoverlapping sequential pattern mining, *Knowledge-Based Systems*, 229 (2021) 107361
 - [41] Y. Wu, R. Lei, Y. Li, L. Guo, X. Wu, HAOP-Miner: Self-adaptive high-average utility one-off sequential pattern mining, *Expert Systems With Applications* 184 (2021) 115449.
 - [42] Y. Wu, X. Liu, W. Yan, L. Guo, X. Wu, Efficient algorithm for solving strict pattern matching under nonoverlapping condition, *Journal of Software*, 32 (11) (2021) 3331–3350.
 - [43] Y. Wu, C. Shen, H. Jiang, X. Wu, Strict pattern matching under non-overlapping condition, *Science China Information Sciences* 60 (1) (2017) 012101.
 - [44] Y. Wu, Z. Tang, H. Jiang, X. Wu, Approximate pattern matching with gap constraints, *Journal of Information Science* 42 (5) (2016) 639–658.
 - [45] Y. Wu, Y. Tong, X. Zhu, X. Wu, NOSEP: Nonoverlapping sequence pattern mining with gap constraints, *IEEE Transactions on Cybernetics* 48 (10) (2018) 2809–2822.
 - [46] Y. Wu, L. Wang, J. Ren, W. Ding, X. Wu, Mining sequential patterns with periodic wildcard gaps, *Applied Intelligence* 41 (1) (2014) 99–116.
 - [47] Y. Wu, Y. Wang, Y. Li, X. Zhu, X. Wu, Top-k self-adaptive contrast sequential pattern mining, *IEEE Transactions on Cybernetics* (2021) DOI: 10.1109/TCYB.2021.3082114
 - [48] Y. Wu, C. Zhu, Y. Li, L. Guo, X. Wu, NetNCSP: Nonoverlapping closed sequential pattern mining, *Knowledge-Based Systems* 196 (2020) 105812.
 - [49] U. Yun, G. Lee, K. H. Ryu, Mining maximal frequent patterns by considering weight conditions over data streams, *Knowledge-Based Systems* 55 (2014) 49–65.
 - [50] M. Zhang, B. Kao, D. W. Cheung, K. Y. Yip, Mining periodic patterns with gap requirement from sequences, *ACM Transactions on Knowledge Discovery from Data* 1 (2) (2007) 7.